```
corpus = [
    "Natural language processing is fun.",
    "Natural language processing is done with machine learning.",
    "Natural language processing is foundational for artificial intelligence.",
    "We use natural language processing to make computers learn.",
    "Computers learn natural language processing to learn human like text.",
    "Artificial intelligence is about self improving computers.",
    "computers teach themselves with artificial intelligence.",
    "machine learning is fun.",
    "artificial intelligence is fun.",
    "it is interesting."
]
```

```
import re
from collections import Counter, defaultdict

class NGramModel:
    def __init__(self, n):
        self.n = n
        self.counts = defaultdict(Counter)
        self.unigram_counts = Counter()
        self.total_words = 0

    def tokenize(self, sentence):
        words = re.findall(r'\b\w+\b', sentence.lower())
        return words

    def fit(self, corpus):
        for sentence in corpus:
            tokens = self.tokenize(sentence)
            if not tokens:
                continue

            if self.n == 1:
                self.unigram_counts.update(tokens)
                self.total_words += len(tokens)
                continue

            padded_tokens = ['<s>'] * (self.n - 1) + tokens

            for i in range(len(padded_tokens) - self.n + 1):
                ngram = padded_tokens[i : i + self.n]
                context = tuple(ngram[:-1])
                target = ngram[-1]
                self.counts[context][target] += 1

    def get_probability(self, word, context=None):
        if self.n == 1:
            if self.total_words == 0:
                return 0.0
            return self.unigram_counts[word] / self.total_words

        if context is None:
            raise ValueError("Context must be provided for n > 1")

        context = tuple(context)
        if len(context) < self.n - 1:
            context = ('<s>',) * (self.n - 1 - len(context)) + context
        elif len(context) > self.n - 1:
            context = context[-(self.n - 1):]

        context_counts = self.counts.get(context, Counter())
        total_context_count = sum(context_counts.values())

        if total_context_count == 0:
            return 0.0

        return context_counts[word] / total_context_count

    def get_predictions(self, context=None, top_k=5):
        if self.n == 1:
            total = sum(self.unigram_counts.values())
            return sorted([(w, count/total) for w, count in self.unigram_counts.items()], key=lambda x: -x[1])[:top_k]

        if context is None:
            return []

        context = tuple(context)
        if len(context) < self.n - 1:
            context = ('<s>',) * (self.n - 1 - len(context)) + context
        elif len(context) > self.n - 1:
            context = context[-(self.n - 1):]

        context_counts = self.counts.get(context, Counter())
        total_context_count = sum(context_counts.values())
        if total_context_count == 0:
            return []

        predictions = [(word, count / total_context_count) for word, count in context_counts.items()]
        return sorted(predictions, key=lambda x: -x[1])[:top_k]
```

```
unigram_model = NGramModel(n=1)
bigram_model = NGramModel(n=2)
trigram_model = NGramModel(n=3)

unigram_model.fit(corpus)
bigram_model.fit(corpus)
trigram_model.fit(corpus)

print("--- Unigram Probabilities (Top 5 words) ---")
print(unigram_model.get_predictions())

print("\n--- Bigram Probabilities following context 'natural' ---")
print(bigram_model.get_predictions(context=('natural',)))

print("\n--- Trigram Probabilities following context 'natural language' ---")
print(trigram_model.get_predictions(context=('natural', 'language')))

print("\n--- Trigram Probabilities following context 'artificial intelligence' ---")
print(trigram_model.get_predictions(context=('artificial', 'intelligence')))
```

```
--- Unigram Probabilities (Top 5 words) ---
[('is', 0.109375), ('natural', 0.078125), ('language', 0.078125), ('processing', 0.078125), ('artificial', 0.0625)]

--- Bigram Probabilities following context 'natural' ---
[('language', 1.0)]

--- Trigram Probabilities following context 'natural language' ---
[('processing', 1.0)]

--- Trigram Probabilities following context 'artificial intelligence' ---
[('is', 1.0)]
```